INSTITUTO FEDERAL DO RIO GRANDE DO NORTE — CAMPUS NATAL ZONA LESTE

Projeto Integrador: Patinhas — Entrega da Etapa 3

Arquitetura Distribuída, API HTTP/REST, Persistência Relacional (SQLite), Controle de Acesso e OpenAPI 3.0

Discente: Lucas Aprígio da Silva | **Matrícula:** 20232158060024 | **Data:** 02/11/2026

🔗 Central de Hiperlinks Oficiais das Entregas (Clique para abrir no navegador)

Item da Entrega	Hiperlink Direto Clicável (GitHub / Swagger UI Online)
1. Repositório Completo (Código-Fonte Integrado)	https://github.com/lucasaprigiodev/projeto-integrador-patinhas
1.1 Servidor API HTTP/REST + JWT + RBAC	https://github.com/lucasaprigiodev/projeto-integrador-patinhas/blob/main/backend/server.js
1.2 Modelagem SQL Relacional (DDL)	https://github.com/lucasaprigiodev/projeto-integrador-patinhas/blob/main/backend/schema.sql
1.3 Camada de Persistência + Seeds (SQLite)	https://github.com/lucasaprigiodev/projeto-integrador-patinhas/blob/main/backend/database.js Arquivo do Banco: https://github.com/lucasaprigiodev/projeto-integrador-patinhas/blob/main/backend/patinhas.sqlite
1.4 Módulo Cliente HTTP Reutilizável (Front-end)	https://github.com/lucasaprigiodev/projeto-integrador-patinhas/blob/main/www/db.js Interfaces Mobile/Web: https://github.com/lucasaprigiodev/projeto-integrador-patinhas/tree/main/www
3. Documentação Interativa OpenAPI (Swagger UI)	https://lucasaprigiodev.github.io/projeto-integrador-patinhas/swagger.html
3.1 Especificação OpenAPI 3.0.3 (JSON)	https://github.com/lucasaprigiodev/projeto-integrador-patinhas/blob/main/backend/openapi.json
4. Aplicativo / Painel Web Executável Online	https://lucasaprigiodev.github.io/projeto-integrador-patinhas/www/01-login.html

1. Diagrama da Arquitetura Distribuída

O sistema **Patinhas** foi estruturado em uma arquitetura cliente-servidor distribuída em três camadas desacopladas. As interfaces de usuário (Aplicativo Mobile Android empacotado com Capacitor e Painel Web Administrativo da ONG) comunicam-se com o servidor exclusivamente por meio do módulo cliente HTTP (`www/db.js`), que realiza a serialização JSON e anexa o token JWT de autenticação para o servidor **API HTTP/REST** (`backend/server.js`), responsável pelas regras de negócio, máquina de estados, controle de acesso (RBAC) e persistência relacional no **SQLite** (`backend/patinhas.sqlite`).

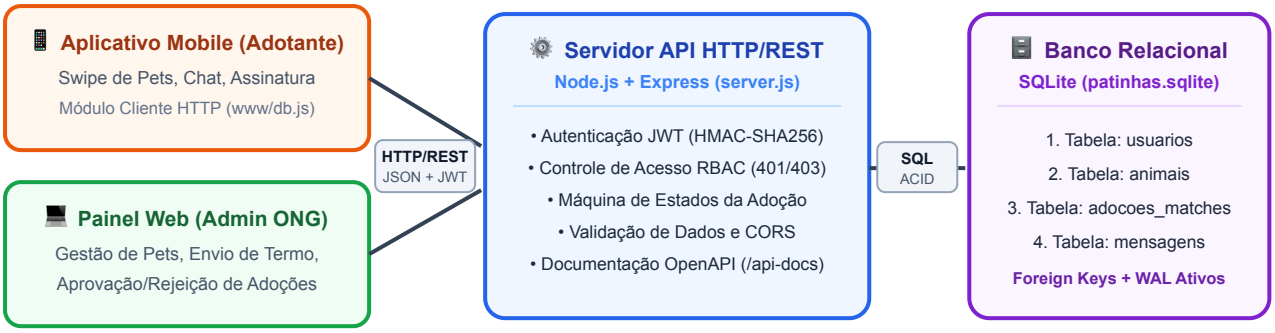


Figura 1 — Arquitetura distribuída do sistema Patinhas: Aplicativo/Painel → API HTTP/REST → Banco de Dados Relacional SQLite.

2. Modelagem e Persistência de Dados Relacional

2.1 Entidades, Relacionamentos e Regras de Integridade

O esquema relacional foi definido em **backend/schema.sql** e instanciado por **backend/database.js** com integridade referencial ativa (`PRAGMA foreign_keys = ON;`):

Entidade (Tabela)	Chaves e Atributos Principais	Relacionamentos (FK)	Regras de Integridade
`usuarios`	`id` (PK), `nome`, `cpf`, `telefone`, `email`, `senha_hash`, `perfil`, `avatar_url`	1:N com `animais` 1:N com `adocoes_matches` 1:N com `mensagens`	`UNIQUE(email)`, `UNIQUE(cpf)`, `CHECK(perfil IN ('adotante','ong'))`, senha com hash `scrypt`.
`animais`	`id` (PK), `nome`, `especie`, `raca`, `idade`, `porte`, `sexo`, `foto_url`, `status`, `ong_id` (FK)	N:1 com `usuarios(id)` via `ong_id` (`ON DELETE RESTRICT`)	`CHECK(status IN ('disponivel', 'em_processo', 'adotado'))`, `CHECK(porte IN ('Pequeno','Médio','Grande'))`.
`adocoes_matches`	`id` (PK), `animal_id` (FK), `adotante_id` (FK), `status`, `termo_enviado`, `termo_assinado`, `assinatura_digital`	N:1 com `animais(id)` N:1 com `usuarios(id)` (`ON DELETE CASCADE`)	`UNIQUE(animal_id, adotante_id)` impede matches duplicados; `CHECK` de estados válidos.
`mensagens`	`id` (PK), `match_id` (FK), `remetente_id` (FK), `remetente_perfil`, `texto`, `is_termo`, `horario`	N:1 com `adocoes_matches(id)` N:1 com `usuarios(id)`	`CHECK(remetente_perfil IN ('user','ong'))`, `CHECK(length(trim(texto)) >= 1)`.

2.2 Justificativa da Escolha do Banco de Dados Relacional (SQLite)

Optou-se pelo banco de dados relacional **SQLite 3** integrado nativamente ao Node.js (módulo `node:sqlite` / `DatabaseSync`) pelos seguintes fatores técnicos:

1. Conformidade Relacional e Transações ACID: O SQLite impõe restrições de chave estrangeira (`FOREIGN KEY`), unicidade (`UNIQUE`) e validação de domínio (`CHECK`) diretamente no nível do SGBD, garantindo integridade total entre usuários, animais, matches e mensagens.

2. Persistência Real em Disco Pós-Reinicialização: Todos os dados são gravados fisicamente no arquivo **backend/patinhas.sqlite**. Ao reiniciar o servidor, todas as contas cadastradas, pets, assinaturas de termos e conversas permanecem intactas.

3. Carga Inicial Automatizada (Seeds): Na primeira execução, `backend/database.js` popula automaticamente 3 usuários iniciais (incluindo a ONG `admin@ong.com` e o adotante `lucas@ifrn.edu.br`), 6 animais e um match em andamento com histórico de chat.

3. Desenvolvimento da API HTTP/REST, Máquina de Estados e REST vs. SOAP

3.1 Máquina de Estados do Processo de Adoção

O fluxo de adoção responsável possui 5 estados controlados pelo servidor, que bloqueia transições inválidas com código `422 Unprocessable Entity` (por exemplo, impedindo que uma adoção seja aprovada antes que o adotante tenha assinado digitalmente o Termo de Adoção Responsável):

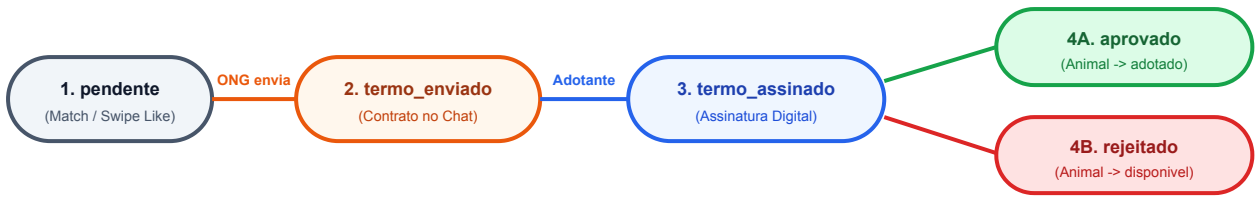


Figura 2 — Máquina de estados finitos do processo de adoção e transições validadas no servidor REST.

3.2 Comparação e Justificativa da Arquitetura REST em Relação ao SOAP

Critério Técnico	Arquitetura REST (Escolhida no Projeto)	Protocolo SOAP
Formato de Mensagem e Rede Mobile	Trafega objetos **JSON** compactos, reduzindo latência e consumo de dados móveis no aplicativo Android.	Exige envelopes **XML** verbosos (`<soap:Envelope>` , `<soap:Body>`), gerando alto overhead de banda.
Semântica do Protocolo HTTP	Utiliza nativamente os métodos HTTP (`GET` , `POST` , `PUT` , `PATCH` , `DELETE`) e códigos HTTP semânticos (`200` , `201` , `400` , `401` , `403` , `404` , `409` , `422`).	Tunela as operações em requisições `POST` genéricas baseadas em `SOAPAction` e retorna erros via `SOAP Fault` .
Consumo nas Interfaces (Front-end)	Consumo direto no JavaScript via `fetch()` e `JSON.parse()` no módulo `www/db.js` , sem dependências pesadas.	Requer leitura de esquemas **WSDL** e clientes complexos de serialização/parsing XML.
Escalabilidade e Estado (Stateless)	Comunicação *stateless* autenticada por token **JWT Bearer** em cada requisição, simplificando o servidor.	Frequentemente acoplado a especificações pesadas como *WS-Security* e controle de sessão complexo.

4. Identificação, Autenticação JWT e Controle de Acesso (RBAC)

O servidor realiza o cadastro (`POST /api/auth/register`) e a autenticação real (`POST /api/auth/login`) consultando a tabela `usuarios`, verificando o hash criptográfico da senha (`crypto.scryptSync`) e emitindo um token **JWT assinado com HMAC-SHA256** contendo `id`, `email` e `perfil` (`adotante` ou `ong`).

- **Acesso sem autenticação (`401 Unauthorized`):** Requisições sem cabeçalho `Authorization: Bearer <token>` ou com token inválido são barradas pelo middleware `authenticateToken`.
- **Permissão insuficiente (`403 Forbidden`):** O middleware `requireRole('ong')` bloqueia usuários do perfil `adotante` que tentem cadastrar/excluir animais, enviar termos de adoção ou aprovar/rejeitar processos. Além disso, cada registro criado (animal, match ou mensagem) fica associado ao `id` do usuário autenticado via chave estrangeira.

5. Documentação da API com OpenAPI 3.0 (Swagger)

A documentação OpenAPI 3.0.3 pode ser explorada interativamente pelo link https://lucasaprigiodev.github.io/projeto-integrador-patinhas/swagger.html ou pelo arquivo JSON em https://github.com/lucasaprigiodev/projeto-integrador-patinhas/blob/main/backend/openapi.json. Abaixo apresenta-se o contrato completo das rotas, regras de validação e exemplos de requisições e respostas documentados:

Método	Rota (Endpoint)	Autenticação / Perfil	Regras de Validação e Códigos HTTP
POST	`/api/auth/register`	Público	Valida `nome` (≥3 chars), `email` único, `senha` (≥4 chars) e `perfil` (`adotante` | `ong`). Retorna `201`, `400` ou `409`.
POST	`/api/auth/login`	Público	Valida `email` e `senha` no SQLite e retorna token JWT. Códigos: `200`, `400`, `401`.
GET	`/api/auth/me`	Bearer JWT (Qualquer)	Retorna os dados do usuário autenticado. Códigos: `200`, `401`.
GET	`/api/pets` e `/api/pets/:id`	Público	Filtra por `?status=disponivel` e `?especie=Cachorro`. Códigos: `200`, `404`.
POST	`/api/pets`	Bearer JWT (`ong`)	Exige `nome`, `raca`, `idade`, `foto_url`; associa ao `ong_id` logado. Códigos: `201`, `400`, `401`, `403`.
PUT / DEL	`/api/pets/:id`	Bearer JWT (`ong`)	Atualiza ou remove animal existente. Códigos: `200`, `401`, `403`, `404`.
GET	`/api/matches`	Bearer JWT (Ambos)	Adotante vê apenas seus matches; ONG vê todos. Códigos: `200`, `401`.
POST	`/api/matches`	Bearer JWT (`adotante`)	Cria match no estado `pendente` para `animal_id` disponível. Códigos: `201`, `404`, `409`, `422`.
POST	`/api/matches/:id/enviar-termo`	Bearer JWT (`ong`)	Transição `pendente` → `termo_enviado` e publica termo no chat. Códigos: `200`, `403`, `404`, `422`.

Método	Rota (Endpoint)	Autenticação / Perfil	Regras de Validação e Códigos HTTP
POST	`/api/matches/:id/assinar-termo`	Bearer JWT (Adotante dono)	Exige `termo_enviado=1` e `assinatura_digital` (≥3 chars). Transição para `termo_assinado`. Códigos: `200`, `400`, `403`, `422`.
PATCH	`/api/matches/:id/status`	Bearer JWT (`ong`)	Transição final para `aprovado` (exige termo assinado) ou `rejeitado`. Códigos: `200`, `400`, `403`, `422`.
GET / POST	`/api/matches/:id/messages`	Bearer JWT (Participantes)	Lista ou envia mensagem vinculada ao `remetente_id` autenticado. Códigos: `200`, `201`, `400`, `403`, `404`.

5.1 Exemplos de Requisições e Respostas (Sucesso e Erro)

```
// EXEMPLO 1: POST /api/auth/login (200 OK – Sucesso)
Request:  { "email": "lucas@ifrn.edu.br", "senha": "123456" }
Response: {
  "mensagem": "Autenticação realizada com sucesso!",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "usuario": { "id": 2, "nome": "Lucas Aprígio", "email": "lucas@ifrn.edu.br", "perfil": "adotante
}

// EXEMPLO 2: POST /api/pets por um usuário com perfil 'adotante' (403 Forbidden – Controle de Ace
Headers:  Authorization: Bearer <token_adotante>
Response: {
  "erro": "Acesso negado. Seu perfil ('adotante') não possui permissão para executar esta operação
  "codigo": 403
}

// EXEMPLO 3: PATCH /api/matches/2/status tentando aprovar sem Termo Assinado (422 Unprocessable E
Request:  { "status": "aprovado" }
Response: {
  "erro": "Transição de estado inválida: não é permitido aprovar a adoção antes da assinatura digi
  "codigo": 422
}
```